

# Lab 5.4: GCP Security Services Baseline

GCP leads with identity and data. Where AWS gives you a flat IAM policy and a Security Hub aggregator, GCP gives you Org Policy that rejects misconfigurations at the API call, Workload Identity Federation that replaces service account keys with short-lived OIDC tokens, and Data Access logs that, deliberately, you have to turn on.

This lab does all three for one project, including the part most guidance skips: a safe way to introduce a preventive control without breaking production on the day you enable it.

---

For reading. Code lines wrap to fit the page; copy code from docs/05\_04\_gcp\_security\_services.md, not the PDF.

**This PDF is for reading.** Long code lines wrap to fit the page, so copy commands and code from `docs/05_04_gcp_security_services.md` in your clone, not from the PDF.

## Learning objectives

- Enforce Org Policy at the API so non-compliant creation is rejected before the resource exists.
- **Roll out a preventive control in audit mode first**, then enforce. This is new in v2.
- Replace long-lived service account keys with Workload Identity Federation.
- Enable Data Access audit logs, the single most-cited GCP audit finding.

## Controls implemented

| Control           | Mechanism                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------|
| CM-6              | <code>storage.uniformBucketLevelAccess</code> , <code>storage.publicAccessPrevention</code> |
| AC-2, IA-5        | <code>iam.disableServiceAccountKeyCreation</code> + WIF                                     |
| AC-3              | <code>compute.requireOsLogin</code>                                                         |
| AU-2, AU-3, AU-12 | Data Access audit logs per service                                                          |
| SC-7              | VPC Service Controls (discussed, not deployed; see Step 6)                                  |
| <b>SC-8</b>       | <b>Inherited from GCP.</b> Cloud Storage and the Google APIs are TLS-only.                  |

Same inherited-control note as Lab 2.4. On AWS you write a bucket policy to get SC-8; on GCP the platform provides it. Record it as `inherited` in your OSCAL rather than claiming you implemented it.

## Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- **Lab 0.1 Part 2** for project, billing, APIs and both authentications. `orgpolicy` and `sts` are the two APIs people forget, and Step 8 enables them.
- A GCP project you own with billing enabled, and APIs `cloudkms`, `iam`, `cloudresourcemanager`, and `orgpolicy` enabled.
- Roles: `roles/orgpolicy.policyAdmin`, `roles/iam.workloadIdentityPoolAdmin`, `roles/logging.admin`. At project scope your project owner can grant these; at org scope you need org-level rights.
- `gcloud auth login` **and** `gcloud auth application-default login`.

- Terraform `>= 1.10`.

The lab uses project scope so it works in any environment. If your project sits in an Organization and you want org or folder scope, the same resources take a different `parent`.

**Read the Cleanup section before you apply this one.** Three things here do not undo the way you would expect:

- **Workload Identity Federation pools enter a 30-day soft delete.** You cannot recreate a pool with the same ID until it expires, unless you undelete and then purge it.
- **Disabling an Org Policy does not retroactively un-enforce it.** Buckets created under it keep the shape it required.
- **Removing an audit config is not retroactive either.** Logs already ingested keep billing until their retention expires.

None of that is a lab artifact. It is what "enforced at the platform" costs, and knowing it before you apply is the difference between a decision and a surprise.

## Estimated time & cost

- 90 minutes, mostly waiting for Org Policy propagation (5 to 10 minutes per change).
- Org Policy: free. WIF: free. Security Command Center Standard: free at org level.
- **Data Access logs: \$0.50/GB ingested plus Cloud Logging storage.** Pennies for an empty project, gigabytes per day in a busy one. Start with one service.

## Architecture

```
project (your-gcp-project)
-----
Org Policy (project scope, REJECT at the API):
  storage.uniformBucketLevelAccess = TRUE    CM-6
  storage.publicAccessPrevention   = TRUE    AC-3
  iam.disableServiceAccountKeyCreation = TRUE  AC-2 / IA-5
  compute.requireOsLogin           = TRUE    AC-3

Workload Identity Federation:
  pool      : github-actions
  provider  : token.actions.githubusercontent.com
  condition: assertion.repository == "GRCEngClub/cgep-app-starter"
  SA        : cgep-grc-gate-sa@... (roles/viewer)
```

```
Data Access audit logs (per service):      AU-2 / AU-3 / AU-12
storage.googleapis.com    DATA_READ + DATA_WRITE + ADMIN_READ
cloudkms.googleapis.com  DATA_READ + DATA_WRITE + ADMIN_READ
iam.googleapis.com        DATA_READ + DATA_WRITE + ADMIN_READ
```

## Step-by-step walkthrough

### Concept: why identity-first

GCP's bet is that the smallest unit of security is the principal, not the resource.

Org Policy enforces at the API call: a bucket creation that violates `uniformBucketLevelAccess` is **rejected**, not flagged. WIF replaces "create a service account, download a JSON key, paste it into GitHub Secrets, hope nobody leaks it" with "the runtime presents an OIDC token, GCP swaps it for a short-lived access token, the token expires by itself."

Together they make whole categories of attack uneconomical. Compare the layers you now have:

| Layer                       | Example                   | When it acts             |
|-----------------------------|---------------------------|--------------------------|
| Preventive, at the API      | Org Policy                | The action never happens |
| Preventive, at the pipeline | Confest gate (Ch 3)       | Before merge             |
| Detective                   | Security Hub, SCC, Config | Minutes to hours after   |

Org Policy is the strongest layer, and it is the one that will page you at 2am if you enable it carelessly. Hence Step 2.

### Step 1 Org Policy in audit mode first

**This is an addition to the official lab, and the most useful habit in it.**

Going straight to `enforce = "TRUE"` is fine in a lab account. In a real project, turning on a rejection at the API without knowing what currently violates it is how you break the deploy pipeline of a team that has never heard of you.

The Org Policy v2 API supports a dry-run specification: the constraint evaluates and logs violations without rejecting anything.

```
# Phase 1: observe. Violations are logged, nothing is blocked.
resource "google_org_policy_policy" "uniform_bucket_access" {
  name   = "projects/${var.gcp_project}/policies/storage.uniformBucketLevelAccess"
  parent = "projects/${var.gcp_project}"
}
```

```

dry_run_spec {
  rules {
    enforce = "TRUE"
  }
}
}

```

Leave it for a week. Query the violations:

```

gcloud logging read \
  'protoPayload.status.details.violations.type="ORG_POLICY_DRY_RUN"' \
  --project="$TF_VAR_gcp_project" --limit=50 --format=json

```

Then, once the list is empty or every entry is understood, promote it:

```

# Phase 2: enforce.
resource "google_org_policy_policy" "uniform_bucket_access" {
  name      = "projects/${var.gcp_project}/policies/storage.uniformBucketLevelAccess"
  parent    = "projects/${var.gcp_project}"

  spec {
    rules {
      enforce = "TRUE"
    }
  }
}

```

**Do not audit by omitting the rules block.** It is a natural guess and it is dangerously wrong: a policy with no rules is not auditing, it is **not in effect at all**. You would observe zero violations and conclude you were compliant. `dry_run_spec` is the mechanism that actually audits. Confirm your `google` provider supports it; it is present in recent 5.x.

The rest of the constraints, in their enforcing form:

```

resource "google_org_policy_policy" "public_access_prevention" {
  name      = "projects/${var.gcp_project}/policies/storage.publicAccessPrevention"
  parent    = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
  }
}

resource "google_org_policy_policy" "disable_sa_keys" {
  name      = "projects/${var.gcp_project}/policies/iam.disableServiceAccountKeyCreation"
  parent    = "projects/${var.gcp_project}"
  spec {
    rules { enforce = "TRUE" }
  }
}

```

```

    }
  }

  resource "google_org_policy_policy" "require_oslogin" {
    name      = "projects/${var.gcp_project}/policies/compute.requireOsLogin"
    parent    = "projects/${var.gcp_project}"
    spec {
      rules { enforce = "TRUE" }
    }
  }
}

```

`storage.publicAccessPrevention` is new in v2 and pairs with Lab 2.4's per-bucket setting. The module sets it on buckets it creates; the org policy sets it on buckets **anyone** creates, including by hand in the console. That is the difference between a module standard and an organizational control, and it is worth saying out loud in your write-up.

## Step 1b Apply it

Nothing above has reached GCP yet. Step 2 tests constraints that do not exist until you apply, and skipping this is easy because Step 1 ends in a wall of HCL rather than a command.

`gcp_project`, `github_org` and `github_repo` have no defaults, because a default project is precisely how you enforce an org policy on the wrong project. Put them in `terraform.tfvars`, which Terraform reads automatically and which is gitignored:

```

# Unquoted heredoc, so the values you set once in cgep.env are written in
# rather than retyped. A tfvars full of placeholders is a tfvars somebody
# eventually commits with a real project id in it.
cat > terraform.tfvars <<EOF
gcp_project = "$TF_VAR_gcp_project"
github_org  = "$TF_VAR_github_org"
github_repo = "$TF_VAR_github_repo"
EOF

```

```

terraform init
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan

```

Apply the **enforcing** form before testing. If `uniform_bucket_access` is still on `dry_run_spec`, Step 2's bucket test will succeed, and it will look like a broken control rather than a policy that is deliberately only watching.

Without the tfvars file Terraform prompts for all three values on every `plan`, `apply` and `destroy`, and fails outright under `-input=false`.

If `cgep.env` from Lab 0.1 Step 15 has `TF_VAR_gcp_project`, `TF_VAR_github_org` and `TF_VAR_github_repo` filled in, skip the file. Source it and Terraform picks them up.

## Step 2 Test the enforcement

```
gcloud iam service-accounts keys create /tmp/key.json \  
  --iam-account="$(terraform output -raw service_account_email)" \  
  --project="$TF_VAR_gcp_project"
```

```
ERROR: (gcloud.iam.service-accounts.keys.create) FAILED_PRECONDITION:  
Key creation is not allowed on this service account.  
constraint iam.disableServiceAccountKeyCreation
```

This is the lesson. The control did not fire after the fact in a finding three hours later. **The action did not happen.** Capture that transcript; it is better evidence than any configuration dump, because it demonstrates enforcement rather than intent.

Do the same for the bucket constraint:

```
# The name carries a random suffix because GCS bucket names are one namespace  
# shared by every Google customer, and this one is meant to be refused rather  
# than to collide with a stranger's.  
TESTBUCKET="cgep-nonuniform-test-$(openssl rand -hex 4)"  
gcloud storage buckets create "gs://$TESTBUCKET" \  
  --project="$TF_VAR_gcp_project" --no-uniform-bucket-level-access  
# expect: rejected by constraint storage.uniformBucketLevelAccess  
  
# If it was NOT rejected, the constraint is not enforcing and you have just  
# created a bucket that allows ACLs. Remove it before moving on.  
gcloud storage buckets delete "gs://$TESTBUCKET" --quiet 2>/dev/null || true
```

## Step 3 Workload Identity Federation

The `attribute_condition` is the whole security of this construct. Without it, **any GitHub repository on the public internet** can present a token to your provider and impersonate your service account.

```
resource "google_iam_workload_identity_pool" "github" {  
  workload_identity_pool_id = "github-actions"  
  display_name              = "GitHub Actions"  
}  
  
resource "google_iam_workload_identity_pool_provider" "github" {  
  workload_identity_pool_id =  
google_iam_workload_identity_pool.github.workload_identity_pool_id  
  workload_identity_pool_provider_id = "github"  
  
  attribute_mapping = {  
    "google.subject"      = "assertion.sub"  
    "attribute.repository" = "assertion.repository"  
    "attribute.actor"     = "assertion.actor"  
    "attribute.ref"       = "assertion.ref"  
  }  
}
```

```

# AC-3. Without this line the provider trusts all of GitHub.
attribute_condition = "assertion.repository == \"${var.github_org}/${var.github_repo}\""

oidc {
  issuer_uri = "https://token.actions.githubusercontent.com"
}

resource "google_service_account" "gha" {
  account_id   = "cgep-grc-gate-sa"
  display_name = "CGE-P GRC gate (read-only)"
}

# AC-6: read-only. The gate plans and inspects; it does not change anything.
resource "google_project_iam_member" "gha_viewer" {
  project = var.gcp_project
  role    = "roles/viewer"
  member  = "serviceAccount:${google_service_account.gha.email}"
}

resource "google_service_account_iam_binding" "wif_user" {
  service_account_id = google_service_account.gha.name
  role                = "roles/iam.workloadIdentityUser"

  members = [
    "principalSet://iam.googleapis.com/${google_iam_workload_identity_pool.github.name}/
    attribute.repository/${var.github_org}/${var.github_repo}",
  ]
}

```

**Defense in depth here is two layers, and students routinely build only one.** The `attribute_condition` on the provider decides who may exchange a token at all. The `principalSet` in the binding decides who may impersonate this particular service account. Set the condition and use `principalSet://.../*` in the binding and you have one layer. Set both and a mistake in either is survivable.

For an apply pipeline, tighten further to a specific ref:

```
principalSet://iam.googleapis.com/P00L/attribute.ref/refs/heads/main
```

The workflow side, keyless:

```

permissions:
  id-token: write
  contents: read

steps:
  - uses: google-github-actions/auth@7c6bc770dae815cd3e89ee6cdf493a5fab2cc093 # v3
    with:
      workload_identity_provider: projects/PROJECT_NUMBER/locations/global/workloadIdentityPools/

```



```
github-actions/providers/github
  service_account: cgep-grc-gate-sa@your-gcp-project.iam.gserviceaccount.com

- run: gcloud storage ls
```

The token is minted at job start, expires in an hour, and never touches disk. Same posture as the AWS OIDC pattern in Lab 4.3, different cloud, identical argument. Note the action is SHA-pinned for the same supply chain reason.

## Step 4 Data Access audit logs

Off by default, and the most-cited GCP audit finding because nobody turns them on.

```
resource "google_project_iam_audit_config" "storage" {
  project = var.gcp_project
  service = "storage.googleapis.com"
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
  audit_log_config { log_type = "ADMIN_READ" }
}

resource "google_project_iam_audit_config" "kms" {
  project = var.gcp_project
  service = "cloudkms.googleapis.com"
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
  audit_log_config { log_type = "ADMIN_READ" }
}

resource "google_project_iam_audit_config" "iam" {
  project = var.gcp_project
  service = "iam.googleapis.com"
  audit_log_config { log_type = "ADMIN_READ" }
  audit_log_config { log_type = "DATA_READ" }
  audit_log_config { log_type = "DATA_WRITE" }
}
```

Verify a read actually lands:

```
gcloud storage ls gs://your-test-bucket
sleep 30
gcloud logging read \
  'protoPayload.serviceName="storage.googleapis.com" AND
  protoPayload.methodName=~"storage.objects.list"' \
  --limit 5 --format=json
```

**Turning them on is AU-2 and AU-12. Reading them is AU-6**, and the same caution from Lab 5.2 applies: a log nobody reviews is a cost center, not a control. Log Analytics or a BigQuery sink is the GCP equivalent of the Athena setup in Lab 5.2. Build one or claim `partial`.

## Step 5 Security Command Center

If your project is in an Organization with SCC enabled, findings flow in automatically. Standard is free at org level; Premium is enterprise-priced and not used here. The lab does not provision SCC because it needs org admin.

```
gcloud scc findings list ORG_ID --source=- --format=json \  
> "$EVIDENCE/scc-findings.json"
```

For a standalone project without an Organization, SCC is unavailable. The Org Policy enforcements above are your preventive layer, and stating that substitution explicitly in your write-up is the right move: "detective coverage is absent at project scope; compensated by preventive enforcement at the API."

## Step 6 What is missing, and naming it

Two real controls this lab does not build, worth knowing so you do not overclaim:

**VPC Service Controls (SC-7).** The genuine data-exfiltration boundary on GCP. A service perimeter stops a principal with valid credentials from copying data to a project outside the perimeter, which IAM alone cannot do. It requires org-level rights and careful planning, and misconfiguring one locks you out of your own project. Name it as a gap with a remediation plan.

**Assured Workloads / CMEK org constraints.** `gcp.restrictNonCmekServices` forces CMEK across services rather than relying on each module to do the right thing. It is the organizational form of what Lab 2.4's module does per-bucket, and it is the same relationship as `storage.publicAccessPrevention` in Step 1.

## Verification

---

**Every check below asserts a value.** The commands this replaces printed a `gcloud org-policies list` and left you to scan it, which cannot fail and so proves nothing. They also spelled your project as a literal `your-gcp-project`, which you would have had to substitute in six places. `TF_VAR_gcp_project` is already exported by `cgep.env`, so the script reads it.

```
bash scripts/verify.sh .
```

**On the last check, which writes.** Proving `iam.disableServiceAccountKeyCreation` works means trying to create a key. If the constraint is missing that attempt succeeds, and you now hold a real long-lived service account private key, on disk and in the project. The script deletes it immediately, reports the failure, and tells you the key id to remove by hand if the rollback itself fails. **A test that can leave a credential behind has to clean up after itself, and has to say so when it cannot.**

This is the opposite of the choice made in Lab 2.4, where the equivalent GCP denial test would have made a bucket public and was replaced with an anonymous read instead. The difference is that a key can be revoked and a public object cannot be un-read.

```

#!/usr/bin/env bash
# scripts/verify.sh
# Verification for Lab 5.4. Every check asserts a value and exits non-zero if
# the project disagrees.
#
# Everything is read as JSON and compared with jq. gcloud's value() projection
# renders booleans with Python's capitalisation and silently omits any field
# name it does not recognize, so a typo and a missing control look the same.
set -uo pipefail

WORKSPACE="${1:-.}"
cd "$WORKSPACE" 2>/dev/null || { echo "no such workspace: $WORKSPACE" >&2; exit 1; }

FAILED=0

pass() { printf ' PASS %-8s %s\n' "$1" "$2"; }
fail() { printf ' FAIL %-8s %s\n' "$1" "$2" >&2; FAILED=1; }

check() { # check CONTROL LABEL EXPECTED ACTUAL
  if [[ "$3" == "$4" ]]; then
    pass "$1" "$2 is $4"
  else
    fail "$1" "$2: expected '$3', got '${4:-(nothing)}'"
  fi
}

PROJECT="${TF_VAR_gcp_project:?set TF_VAR_gcp_project, or source cgep.env}"
SA=$(terraform output -raw service_account_email)
PROVIDER=$(terraform output -raw workload_identity_provider)
MODE=$(terraform output -raw enforce_mode)

# projects/NUMBER/locations/global/workloadIdentityPools/POOL/providers/NAME
POOL=$(awk -F/ '{print $6}' <<<"$PROVIDER")

# enforce writes spec; dry_run writes dryRunSpec. Asserting the wrong one
# passes a project where nothing is actually enforced.
if [[ "$MODE" == "enforce" ]]; then
  SPEC=".spec"
else
  SPEC=".dryRunSpec"
fi

echo "=== org policy constraints (mode: $MODE) ==="

for constraint in storage.uniformBucketLevelAccess storage.publicAccessPrevention \
  iam.disableServiceAccountKeyCreation compute.requireOsLogin; do
  check ORG "$constraint" "true" \
    "$(gcloud org-policies describe "$constraint" --project="$PROJECT" --format=json 2>/dev/null \
      | jq -r "${SPEC}.rules[0].enforce // empty")"
done

echo "=== data access logging ==="

# AU-3 and AU-12. ADMIN_READ is on by default; DATA_READ and DATA_WRITE are
# the ones that cost money and are therefore the ones people quietly skip.
IAM=$(gcloud projects get-iam-policy "$PROJECT" --format=json 2>/dev/null)

```

```

[[ -z "$IAM" ]] && IAM='{}'
for service in storage.googleapis.com cloudkms.googleapis.com iam.googleapis.com; do
  for logtype in ADMIN_READ DATA_READ DATA_WRITE; do
    check AU-12 "$service $logtype" "$logtype" \
      "$(jq -r --arg s "$service" --arg t "$logtype" \
        '[.auditConfigs[]? | select(.service==$s) | .auditLogConfigs[]? | select(.logType==$t)
| .logType][0] // empty' \
        <<<"$IAM")"
  done
done

echo "=== workload identity ==="

check AC-3 "pool state" "ACTIVE" \
  "$(gcloud iam workload-identity-pools describe "$POOL" --location=global \
    --project="$PROJECT" --format=json 2>/dev/null | jq -r '.state // empty')")

# AC-6. The gate service account must not carry primitive roles. Anything
# matching roles/owner, roles/editor or roles/viewer here defeats the point of
# a purpose-built identity.
PRIMITIVE=$(jq -r --arg sa "serviceAccount:$SA" \
  '[.bindings[]? | select(.members[]? == $sa) | .role
| select(. == "roles/owner" or . == "roles/editor" or . == "roles/viewer")] | length' \
  <<<"$IAM")
check AC-6 "primitive roles on gate SA" "0" "$PRIMITIVE"

echo "=== enforcement ==="

# The service account key constraint, actually refusing.
#
# This one attempts a WRITE, which needs care: if the constraint is missing the
# attempt succeeds and leaves a real, long-lived private key on disk and in the
# project. So it is cleaned up immediately and reported as a failure, which is
# what it is.
KEYFILE=$(mktemp /tmp/cgep-sa-key.XXXXXX.json)
if gcloud iam service-accounts keys create "$KEYFILE" \
  --iam-account="$SA" --project="$PROJECT" >/dev/null 2>&1; then
  KEY_ID=$(jq -r '.private_key_id // empty' "$KEYFILE" 2>/dev/null)
  if [[ -n "$KEY_ID" ]]; then
    gcloud iam service-accounts keys delete "$KEY_ID" --iam-account="$SA" \
      --project="$PROJECT" --quiet >/dev/null 2>&1 \
      && fail "AC-6" "key creation SUCCEEDED and was rolled back. The constraint is not enforcing." \
      || fail "AC-6" "key creation SUCCEEDED and could not be rolled back. Delete key
$KEY_ID by hand, now."
  else
    fail "AC-6" "key creation SUCCEEDED. The constraint is not enforcing."
  fi
else
  pass "AC-6" "service account key creation was refused"
fi
rm -f "$KEYFILE"

echo
if [[ $FAILED -eq 0 ]]; then
  echo "VERIFIED: every control asserted above holds in the project."

```

```

else
  echo "NOT VERIFIED: at least one control does not hold. Do not capture evidence yet." >&2
fi
exit $FAILED

```

## Capture the evidence the checklist asks for

```

# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-5-4"
mkdir -p "$EVIDENCE"
gcloud projects get-iam-policy "$TF_VAR_gcp_project" --format=json \
  > "$EVIDENCE/iam-policy.json"

{
  echo "### service account key creation, expect FAILED_PRECONDITION"
  gcloud iam service-accounts keys create /tmp/key.json \
    --iam-account="$SA_EMAIL" --project="$TF_VAR_gcp_project" 2>&1
} | tee "$EVIDENCE/enforcement-denied.txt"

```

## Portfolio submission checklist

- ☐ terraform/baselines/gcp/ committed.
- ☐ At least one workflow using WIF. **No service account JSON key anywhere in the repo or its history.**
- ☐ evidence/lab-5-4/iam-policy.json capturing the Data Access log configuration.
- ☐ evidence/lab-5-4/enforcement-denied.txt, the transcripts of both rejected actions.
- ☐ README notes the "Data Access logs are off by default" lesson, and names VPC Service Controls as a known gap.

## Troubleshooting

- **Org Policy propagation latency.** First-apply changes take 5 to 10 minutes. A test run immediately after terraform apply may briefly succeed. That is propagation, not a broken policy.
- **PERMISSION\_DENIED on the WIF provider.** You need roles/iam.workloadIdentityPoolAdmin specifically. Being project Owner is not sufficient.
- **attribute.repository mismatch.** GitHub's assertion.repository is the literal OWNER/REPO. Case, spelling, and the slash all matter. An opaque PERMISSION\_DENIED from auth@v2 is almost always this.
- **policySpec is not supported.** Enable the v2 API: gcloud services enable orgpolicy.googleapis.com.
- **dry\_run\_spec unrecognized.** Your google provider is too old. Upgrade, or accept that you are enforcing without an observation period and say so.
- **Data Access log cost.** In a busy project these ingest gigabytes per day. Start with storage.googleapis.com alone before adding KMS and IAM.

## Cleanup

---

```
terraform destroy -auto-approve
```

Three things that will surprise you:

1. **WIF pools enter a 30-day soft delete.** You cannot recreate a pool with the same ID until it expires, or you `gcloud iam workload-identity-pools undelete` and then delete again with `--purge`.
2. **Disabling Org Policy does not retroactively un-enforce.** Buckets created with uniform access stay that way. The policy only stops blocking new violations.
3. **Audit config removal is not retroactive either.** Logs already ingested continue to bill for storage until their retention expires.

## How this feeds the capstone

---

The WIF pattern is your AWS-OIDC equivalent for anything GCP-touching. If your capstone pipeline reaches into GCP for any reason, it uses WIF, never a key.

The Org Policy layer is the preventive tier above your Rego, which is detective. Saying that clearly in your write-up, with the three-layer table from the Concept section, is one of the strongest paragraphs you can write about defense in depth, because it shows you know which of your controls acts when.

The Data Access logs feed your OSCAL component's AU-2 and AU-3 statements: enabled per service, with the IAM policy JSON as evidence. And the audit-mode-then-enforce rollout in Step 1 is the answer to the write-up question "how would you introduce this control without breaking the engineering team," which the capstone brief asks in the form "without slowing the engineering team down."

## Revision history

---

### These notes

- Org Policy constraints roll out in `dry_run_spec` first and promote to `spec`, so a preventive control can be introduced without breaking a team that has never heard of you.
- Corrected the audit-mode guidance: omitting the rules block does not audit, it leaves the policy not in effect at all.
- Added `storage.publicAccessPrevention` alongside the original three constraints.
- Workload Identity Federation documented as two layers, the provider `attribute_condition` and the binding `principalSet`, rather than one.
- Named VPC Service Controls and CMEK org constraints as known gaps rather than leaving them unmentioned.

## **The official labs**

Initial release: three constraints, straight to enforce, WIF documented as a single layer.